

</>

The Anuj DECODED

</>

```
01 #include <iostream>
02 #include <vector>
03 #include <unordered_map>
04 using namespace std;
05
06 class Solution {
07 public:
08     vector<int> twoSum(
09         vector<int>& nums,
10         int target) {
11         unordered_map<int, int> mp;
12         for (int i = 0; i < nums.size(); ++i) {
13             int diff = target - nums[i];
14             if (mp.count(diff)) {
15                 return {mp[diff], i};
16             }
17             mp[nums[i]] = i;
18         }
19         return {};
20     }
};
```



```
01 // Think Deeper
02 // Write Cleaner
03 // Solve Smarter
04 // Get Stronger
05 // Every day
06 // a better
07 // problem solver
```

CODE. SOLVE. **IMPROVE**. REPEAT.



THINK
DEEPER



WRITE
CLEANER



SOLVE
SMARTER



GET
STRONGER

“

Problem today,
Stronger tomorrow.

This Notebook Belongs To:

ANUJ

940 · Distinct Subsequences II

LeetCode · Classic Hard DP

Hard

String

Dynamic Programming

1. What Type of Question Is This?

This is a **single-string counting DP** problem: count how many distinct non-empty subsequences a string has, modulo 10^9+7 . The tricky part isn't building a recurrence — it's **avoiding double-counting**. A naive "every subsequence can be extended by every future character" recurrence overcounts massively whenever a character repeats, because the exact same subsequence can be reproduced by picking a different occurrence of a repeated letter. The whole problem is about finding a clean way to subtract exactly those duplicates.

2. What Is Given?

- A string **s** of lowercase English letters.
- A **subsequence** is formed by deleting zero or more characters without disturbing the relative order of what remains (e.g. "ace" is a subsequence of "abcde"; "aec" is not, since that reorders characters).
- Two subsequences are **distinct** if they read as different strings — unlike Problem 115 (Distinct Subsequences I), here it's the resulting *text* that must differ, not the positions deleted.

3. What Do We Need to Find?

The number of **distinct non-empty** subsequences of **s**, returned **modulo $10^9 + 7$** since the true count can be astronomically large (up to roughly 2^{2000}).

Function Signature

■ C++

```
class Solution {
public:
    int distinctSubseqII(string s) {

    }
};
```

Examples Walkthrough

Example 1 — used as the running example for every code sample below

```
Input: s = "abc"
Output: 7
```

All characters are distinct, so every one of the $2^3 = 8$ possible subsequences (including the empty one) is itself distinct as text. Excluding the empty subsequence: "a", "b", "c", "ab", "ac", "bc", "abc" → 7.

Example 2 — a repeated letter

```
Input: s = "aba"
Output: 6
```

The distinct subsequences are "a", "b", "ab", "aa", "ba", "aba" → 6. Note there is only *one* "a" in the list even though the letter 'a' appears twice in s — both occurrences of 'a' alone produce the exact same one-character text, so they only count once.

Example 3 — all identical letters

```
Input: s = "aaa"
Output: 3
```

Only "a", "aa", and "aaa" are distinct as text — there is exactly one way to choose "pick k of the three a's" per length that produces a genuinely different result, since every choice of k a's spells the same string → 3.

Constraints

- $1 \leq s.length \leq 2000$
- s consists of lowercase English letters.

The raw count of subsequences can be as large as 2^{2000} , so every intermediate computation must be taken modulo 10^9+7 — including being careful with subtraction, which can go negative before the modulo in several languages.

Overall Workflow & How to Think About It

Step-by-step reasoning to arrive at the right approach:

- **Step 1 — Try the obvious recurrence first.** If total is "the number of distinct subsequences using characters seen so far", then appending a new character c seems to double the count: every previous subsequence can either exclude c (stay as is) or include c appended at the end. That suggests $\text{total} \rightarrow 2 \times \text{total}$, plus the brand-new one-character subsequence " c " itself.
- **Step 2 — Notice why that overcounts.** If c appeared earlier in the string too, then "take an old subsequence and append this new c " can reproduce a text that was *already produced* the last time c appeared and got appended to the same set of earlier subsequences. Those duplicates need to be subtracted back out.
- **Step 3 — Identify exactly what to subtract.** The subsequences that get duplicated are precisely the ones that were newly created the *previous* time this same character was appended — because appending c again after an identical earlier prefix reproduces exactly that batch of strings. So each character needs to remember how much it contributed last time it appeared.
- **Step 4 — Turn that into a per-character running total.** Keep a bucket per letter (26 buckets, one per lowercase letter) that accumulates exactly how many distinct subsequences have ever been counted *because* they ended in that letter. When the letter appears again, subtract its own bucket before adding the new contribution, so nothing already attributed to that letter gets counted twice.
- **Step 5 — Express the new contribution at each position explicitly.** At position i with character c : the number of *brand-new* distinct subsequences ending exactly at position i is 1 (c by itself) plus every subsequence formed from characters before i (call that running sum), minus whatever has already been credited to letter c 's bucket from earlier occurrences. Add each position's new contribution to both the overall total and to c 's bucket.

Which Algorithm Is Best?

Once Step 5's per-position formula is in place, one linear pass over s computes the answer directly — $O(n)$ time using only a length-26 array as auxiliary state. There is no need for a full $O(n^2)$ or even $O(26n)$ table; every position's contribution is an $O(1)$ update given the running sum and the one relevant letter bucket. The brute-force and "better" tiers below build up to this by first enumerating subsequences directly, then computing the same idea with an explicit resum every step, before collapsing to the single-pass version.

Approach 1 — Brute Force (Enumerate Every Subsequence, Deduplicate with a Set)

Idea: Generate every possible subsequence by recursively choosing to skip or take each character, collect the resulting strings into a set so duplicates are automatically merged, and return how many distinct non-empty strings ended up in the set.

Steps

- Recurse with (index, builtSoFar): at each character, branch into "skip it" and "take it" (append it to builtSoFar).
- When the recursion reaches the end of s, insert builtSoFar into a set, as long as it's non-empty.
- After exploring every branch, the set's size is exactly the count of distinct non-empty subsequences.
- Return that size modulo 10^9+7 .

Approach 1 — Code (all languages run Example 1)

■ C++

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    set<string> seen;

    void gen(string& s, int i, string cur) {
        if (i == (int)s.size()) {
            if (!cur.empty()) seen.insert(cur);
            return;
        }
        gen(s, i + 1, cur);           // skip s[i]
        gen(s, i + 1, cur + s[i]);    // take s[i]
    }

    int distinctSubseqII(string s) {
        seen.clear();
        gen(s, 0, "");
        long long MOD = 1000000007;
        return (int)((long long)seen.size() % MOD);
    }
};

int main() {
    string s = "abc";
    Solution sol;
    cout << sol.distinctSubseqII(s) << endl;    // Expected output: 7
    return 0;
}
```

■ Python

```
class Solution:
    def distinctSubseqII(self, s):
        MOD = 10**9 + 7
        seen = set()

        def gen(i, cur):
            if i == len(s):
                if cur:
                    seen.add(cur)
                return
            gen(i + 1, cur)           # skip s[i]
            gen(i + 1, cur + s[i])    # take s[i]
```

```

        gen(0, "")
        return len(seen) % MOD

if __name__ == "__main__":
    s = "abc"
    print(Solution().distinctSubseqII(s))    # Expected output: 7

```

■ JavaScript

```

class Solution {
    distinctSubseqII(s) {
        const MOD = 1000000007;
        const seen = new Set();

        const gen = (i, cur) => {
            if (i === s.length) {
                if (cur.length > 0) seen.add(cur);
                return;
            }
            gen(i + 1, cur);           // skip s[i]
            gen(i + 1, cur + s[i]);    // take s[i]
        };

        gen(0, "");
        return seen.size % MOD;
    }
}

const s = "abc";
console.log(new Solution().distinctSubseqII(s)); // Expected output: 7

```

Complexity: There are 2^n possible skip/take branches, and building/inserting each resulting string costs up to $O(n)$ — $O(n \cdot 2^n)$ time and $O(2^n)$ space in the worst case. With n up to 2000 this is astronomically infeasible (2^{2000} is far beyond any computer's reach); this only works for illustrative inputs of a handful of characters.

Approach 2 — Better (Per-Letter DP Bucket, Resummed Every Step)

Idea: Apply Steps 3–4 of the workflow directly: keep one bucket per letter of the alphabet holding "distinct subsequences ending in this letter", and whenever a letter reappears, simply **overwrite** its bucket with a freshly computed value instead of adding to it — overwriting is what prevents the old contribution from being double counted. The running total needed for that computation is recomputed from scratch (summing all 26 buckets) at every character.

Steps

- Keep a 26-length array `endCount`, all zeros initially.
- For each character `ch` in `s`: sum all 26 buckets to get the current total distinct subsequence count.
- `newVal = total + 1` (every existing subsequence extended by `ch`, plus `ch` alone).
- Overwrite `endCount[ch] = newVal` — this replaces whatever `ch` contributed last time, so nothing is double counted.
- After processing all of `s`, sum the 26 buckets one final time for the answer.

Approach 2 — Code (all languages run Example 1)

■ C++

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int distinctSubseqII(string s) {
        const long long MOD = 1000000007;
        vector<long long> endCount(26, 0); // endCount[c] = distinct subsequences ending in c

        for (char ch : s) {
            long long total = 0;
            for (int c = 0; c < 26; c++) total = (total + endCount[c]) % MOD; // resum all 26 every step

            long long newVal = (total + 1) % MOD; // extend every subsequence so far, plus ch alone
            endCount[ch - 'a'] = newVal;
        }

        long long answer = 0;
        for (int c = 0; c < 26; c++) answer = (answer + endCount[c]) % MOD;
        return (int)answer;
    }
};

int main() {
    string s = "abc";
    Solution sol;
    cout << sol.distinctSubseqII(s) << endl; // Expected output: 7
    return 0;
}
```

■ Python

```
class Solution:
    def distinctSubseqII(self, s):
        MOD = 10**9 + 7
        endCount = [0] * 26 # endCount[c] = distinct subsequences ending in c

        for ch in s:
            total = sum(endCount) % MOD # resum all 26 every step
            newVal = (total + 1) % MOD
```

```

        endCount[ord(ch) - ord('a')] = newVal

        return sum(endCount) % MOD

if __name__ == "__main__":
    s = "abc"
    print(Solution().distinctSubseqII(s))    # Expected output: 7

```

■ JavaScript

```

class Solution {
    distinctSubseqII(s) {
        const MOD = 1000000007;
        const endCount = new Array(26).fill(0);    // endCount[c] = distinct subsequences ending in c

        for (const ch of s) {
            let total = 0;
            for (let c = 0; c < 26; c++) total = (total + endCount[c]) % MOD;    // resum all 26 every step

            const newVal = (total + 1) % MOD;
            endCount[ch.charCodeAt(0) - 97] = newVal;
        }

        let answer = 0;
        for (let c = 0; c < 26; c++) answer = (answer + endCount[c]) % MOD;
        return answer;
    }
}

const s = "abc";
console.log(new Solution().distinctSubseqII(s));    // Expected output: 7

```

Complexity: Summing 26 buckets happens once per character, so this is $O(26n)$ time overall — with n up to 2000 that's only about 52,000 operations, comfortably fast in practice, but it repeats a full 26-wide summation at every single step even though only one bucket actually changed last time. $O(26)$ extra space for the bucket array.

Approach 3 — Optimal (Single Pass, $dp[i]$ = New Subsequences Ending Exactly at i)

Idea: Reformulate Step 5 so nothing is ever resummed. Define $dp[i]$ as the number of **brand-new** distinct subsequences that end exactly at position i , and maintain **two running accumulators alongside it**: $total$ (the sum of all dp values so far) and $countEndWith[c]$ (the sum of every $dp[i]$ where $s[i]$ was letter c). Because both accumulators are updated incrementally by simply adding the newest $dp[i]$, the per-letter 26-wide resummation from Approach 2 disappears entirely — each position does a fixed, constant amount of work.

Steps

- Keep $total = 0$ and $countEndWith[26] = \{0, \dots, 0\}$.
- For each position i with character $s[i] = c$: $dp[i] = 1 + total - countEndWith[c]$ (the $+1$ is the one-character subsequence " c " itself; $total$ accounts for extending every subsequence seen so far; subtracting $countEndWith[c]$ removes exactly the subsequences that were already credited to letter c the last time it appeared, so they aren't recounted).
- Add $dp[i]$ into $total$ ($total$ now reflects every distinct subsequence counted through position i).
- Add $dp[i]$ into $countEndWith[c]$ as well, so the *next* time c appears, its cumulative prior contribution is fully known and can be subtracted cleanly.
- After the loop, $total$ is the final answer — no separate "subtract the empty subsequence" step needed, since $dp[i]$ never counts the empty subsequence in the first place.

Approach 3 — Code (all languages run Example 1)

■ C++

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int distinctSubseqII(string s) {
        int n = s.size();
        int MOD = 1e9 + 7;

        vector<int> dp(n, 1);
        vector<int> countEndWith(26, 0);

        int sum = 0;

        for (int i = 0; i < n; i++) {
            int idx = s[i] - 'a';
            dp[i] = (1 + sum - countEndWith[idx] + MOD) % MOD;

            sum = (sum + dp[i]) % MOD;
            countEndWith[idx] = (countEndWith[idx] + dp[i]) % MOD;
        }

        return sum;
    }
};

int main() {
    string s = "abc";
    Solution sol;
    cout << sol.distinctSubseqII(s) << endl;    // Expected output: 7
    return 0;
}
```

■ Python

```

class Solution:
    def distinctSubseqII(self, s):
        MOD = 10**9 + 7
        n = len(s)

        dp = [0] * n
        countEndWith = [0] * 26
        total = 0

        for i in range(n):
            idx = ord(s[i]) - ord('a')
            dp[i] = (1 + total - countEndWith[idx]) % MOD

            total = (total + dp[i]) % MOD
            countEndWith[idx] = (countEndWith[idx] + dp[i]) % MOD

        return total

if __name__ == "__main__":
    s = "abc"
    print(Solution().distinctSubseqII(s))    # Expected output: 7

```

■ JavaScript

```

class Solution {
    distinctSubseqII(s) {
        const MOD = 1000000007;
        const n = s.length;

        const dp = new Array(n).fill(0);
        const countEndWith = new Array(26).fill(0);
        let total = 0;

        for (let i = 0; i < n; i++) {
            const idx = s.charCodeAt(i) - 97;
            dp[i] = ((1 + total - countEndWith[idx]) % MOD + MOD) % MOD;

            total = (total + dp[i]) % MOD;
            countEndWith[idx] = (countEndWith[idx] + dp[i]) % MOD;
        }

        return total;
    }
}

const s = "abc";
console.log(new Solution().distinctSubseqII(s));    // Expected output: 7

```

Complexity: A single $O(n)$ pass, with $O(1)$ work per character (a handful of array lookups and mod arithmetic) — no inner loop over the alphabet at all. $O(26)$ extra space for the `countEndWith` array plus $O(n)$ for the `dp` array (which could even be reduced to a single scalar if you don't need to inspect `dp[i]` values individually afterward). This comfortably handles the full $n = 2000$ constraint many times over.

Side-by-Side Comparison

1. Brute Force	Enumerate every subsequence via recursion, dedupe with a set, and return the answer. $O(n \cdot 2^n)$ — finishes at $n=2000$		
2. Better	26-bucket DP, resum all buckets from scratch every character. $O(26n)$		$O(26)$
3. Optimal	Single pass; $dp[i]$ via running total and per-letter cumulative bucket. $O(n)$		$O(26) + O(n)$ for $dp[]$

Key Takeaways to Remember

- **"Count distinct subsequences/strings" almost always hides a double-counting trap** around repeated characters — identify precisely what gets duplicated before writing any recurrence.
- A bucket **overwritten** (not added to) per repeated element, tracking "what this element contributed last time", is a general pattern for exactly this kind of duplicate-avoidance — it shows up across many distinct-subsequence and distinct-subset counting problems.
- When a recurrence needs "the sum of some accumulator so far", maintaining that sum **incrementally** (add the newest term each step) is almost always better than recomputing it via a fresh loop — the same $O(26n) \rightarrow O(n)$ pattern shows up constantly whenever a per-category resum can be replaced with a running total.
- Always apply the modulo after subtraction with a $+MOD$ safety margin $((x - y + MOD) \% MOD)$ — plain $(x - y) \% MOD$ can return a negative result in several languages when $x < y$.
- All three tiers were validated against **Example 1** ($s = \text{"abc"} \rightarrow 7$) across C++, Python, and JavaScript, and checked against Examples 2 and 3 ($\text{"aba"} \rightarrow 6$, $\text{"aaa"} \rightarrow 3$) as well.